

HAI822I - EXERCICES SUR :  
LES ASSERTIONS ET LES EXCEPTIONS EN JAVA  
LES PILES

Observer la description du type abstrait de données *Pile* :

- le type défini est *Pile*
- Opérations
  - initialiser()
  - empiler(T element)
  - depiler() : T
  - sommet() : T
  - estVide() : boolean
- Préconditions
  - précondition (p.depiler()) :  $\neg p.estVide()$
  - précondition (p.sommet()) :  $\neg p.estVide()$
- Axiomes
  - Juste après l'instruction *p.empiler(t)*, on a *p.sommet() = t*
  - Juste après *p.initialiser()*, on a *p.estVide() = true*
  - Juste après *p.empiler(t)*, on a *p.estVide() = faux*
  - Juste après *p.empiler(t)*, on a *p.depiler() = t*

Une interface *IPile* représentant le type abstrait *Pile*, une classe *Pile* l'implémentant avec une *ArrayList* pour stocker les éléments et une classe exception *PileVideException* sont présentées ci-dessous.

```
public class PileVideException extends Exception {
    public PileVideException() { }
    public PileVideException(String message) { super(message); }
}
```

```
public interface IPile<T>
{
    void initialiser();
    void empiler(T t) throws PileVideException;
    T depiler()throws PileVideException;;
    T sommet() throws PileVideException;
    boolean estVide();
    int taille();
}
```

```
public class Pile<T> implements IPile<T>{
    // structure de stockage interne des éléments
    private ArrayList<T> elements;

    // Mise en oeuvre des opérations
    public Pile(){initialiser();}

    public T depiler() throws PileVideException{
        if (this.estVide())
            throw new PileVideException("en dépilant");
        T sommet = elements.get(elements.size()-1);
        elements.remove(sommet);
        return sommet;
    }
}
```

```

    }

    public void empiler(T t) throws PileVideException {
        elements.add(t);
        assert this.sommet()==t : "dernier empile =" + this.sommet();
    }

    public boolean estVide() {return elements.isEmpty();}

    public void initialiser() {elements = new ArrayList<T>();}

    public T sommet() throws PileVideException{
        if (this.estVide())
            throw new PileVideException("en dépiler");
        return elements.get(elements.size()-1);
    }

    public int taille(){return elements.size();}
    public String toString(){return "Pile = " + elements;}
}

```

QUESTION 1 Vous pouvez observer la manière dont la spécification est mise en œuvre, en particulier concernant les axiomes. La déclaration `public void empiler(T t) throws PileVideException` peut paraître surprenante puisque dans la définition du type abstrait l'opération `empiler` n'a pas de pré-condition. Voyez-vous la raison pratique qui l'explique ? Peut-on écrire l'assertion autrement pour éviter la déclaration `throws PileVideException` ?

QUESTION 2 Ajouter des assertions pour contrôler que :

- Après avoir construit la pile, celle-ci est vide.
- À la fin de l'opération `depiler`, le nombre d'éléments de la pile a diminué de 1.
- À la fin de l'opération `empiler`, le nombre d'éléments de la pile a augmenté de 1.

QUESTION 3 On désire créer un nouveau type de pile, représentant les piles bornées, c'est-à-dire des piles dont le nombre d'éléments doit rester inférieur à une certaine limite (taille maximale). Cette taille maximale peut évoluer pendant la vie de la pile.

- Proposez une description du type abstrait (en indiquant en quoi il diffère du type pile).
- Ecrivez les classes d'exceptions représentant les erreurs qui peuvent se produire sur les piles bornées : la pile est pleine et on ne peut plus ajouter d'éléments ; la taille maximale ne peut pas être négative ; la taille maximale est diminuée mais le nombre d'éléments déjà stockés est supérieur à cette nouvelle taille maximale.
- Proposez une interface pour représenter le type *Pile bornée*.
- Proposez une classe pour l'implémenter. Les méthodes comporteront des assertions et des signalements d'exceptions aux endroits nécessaires. Par défaut, la taille maximale est de 10 éléments.
- Ecrivez un programme utilisant cette classe. Est-ce qu'une pile bornée peut remplacer une pile : lors de la compilation (substituabilité syntaxique) ? lors de l'exécution (substituabilité comportementale) ?